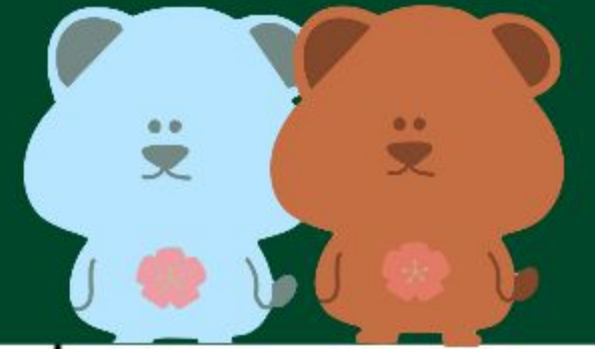




분류 - Part1

3팀 진웨이안, 김서연, 박린

목차

#01 분류(Classification), 결정 트리(Decision Tree)

#02 앙상블 학습(Ensemble Learning)

#03 랜덤 포레스트(Random Forest)



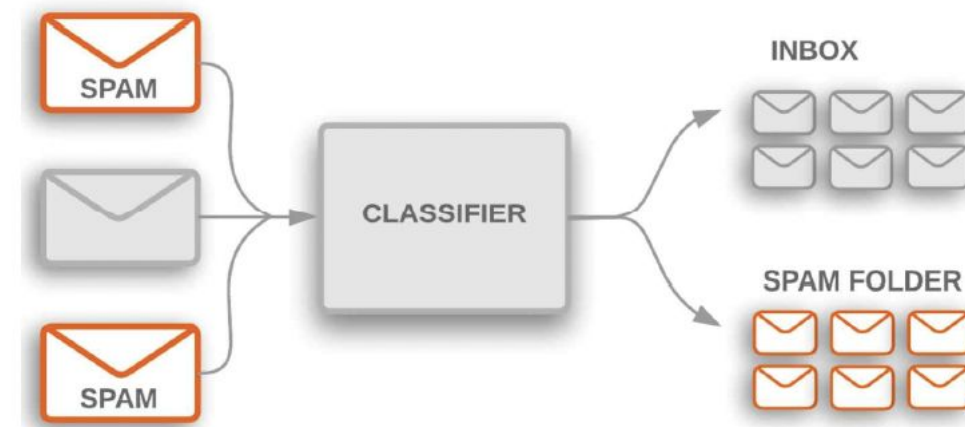
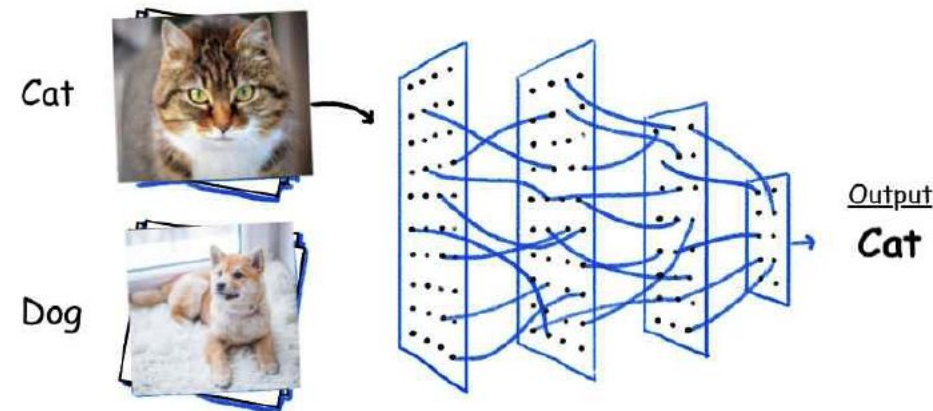
#1.1 분류(Classification)



#1.1 분류의 개요

❖ 분류(Classification)란?

- ✓ 지도 학습(Supervised Learning)의 대표적인 유형
- ✓ 정답이 있는 데이터 세트(Labeled Dataset)을 이용해 모델을 학습
- ✓ 학습한 모델을 사용해 보지 못한 데이터의 레이블을 예측
- ✓ 예:
 - 사진 속 동물 분류
 - 이메일이 스팸인지 아닌지 구분 등

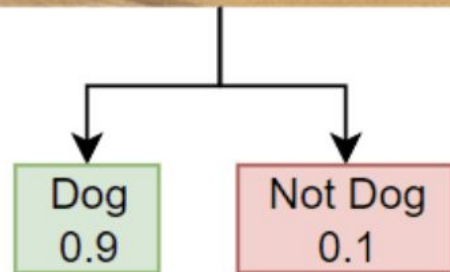


#1.1 분류의 개요

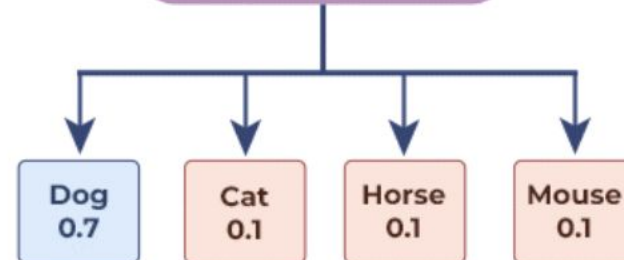
◆ 분류의 종류 (Types of Classification)

- ✓ 이진분류 (Binary Classification)
- ✓ 다중 클래스 분류 (Multiclass Classification)
- ✓ 다중 레이블 분류 (Multi-Label Classification)

Binary Classification



Multiclass Classification



Classes

(pick one class)

- ☒ Dog
- ☐ Cat
- ☐ Horse
- ☐ Mouse

Multi-label Classification



- Dog
- Cat
- Horse
- Fish
- Bird

#1.1 분류의 개요

◆ 분류를 구현할 수 있는 알고리즘

- ✓ 나이브 베이즈 (Naïve Bayes)
 - 조건부 확률을 기반으로 빠르게 분류하는 간단한 알고리즘
- ✓ 로지스틱 회귀 (Logistic Regression)
 - 이진 분류에 주로 사용되는 선형적 확률 모델
- ✓ 결정 트리 (Decision Tree)
 - 질문과 답변 형태의 규칙을 학습해 데이터를 분류하는 트리 구조의 알고리즘
- ✓ 서포트 벡터 머신 (Support Vector Machine)
 - 데이터를 구분하는 최적의 경계면을 찾아 분류하는 알고리즘
- ✓ 최소 근점 알고리즘 (Nearest Neighbor)
 - 가장 가까운 데이터의 레이블로 미지의 데이터를 분류하는 알고리즘
- ✓ 신경망 (Neural Network)
 - 인간의 뇌 구조에서 영감을 받아 설계된 복잡한 패턴 분류 알고리즘
- ✓ 앙상블 (Ensemble)

#1.2 결정 트리(Decision Tree)



#1.2 결정 트리(Decision Tree)

❖ 앙상블이란?

- ✓ 여러 모델의 예측을 결합하여 더 나은 성능을 얻는 방법
- ✓ 예: 랜덤 포레스트(Random Forest), 그라디언트 부스팅(Gradient Boosting)
- ✓ 대부분 동일한 알고리즘을 여러 개 결합해서 사용
- ✓ 기본적으로 사용하는 알고리즘은 결정 트리(Decision Tree)

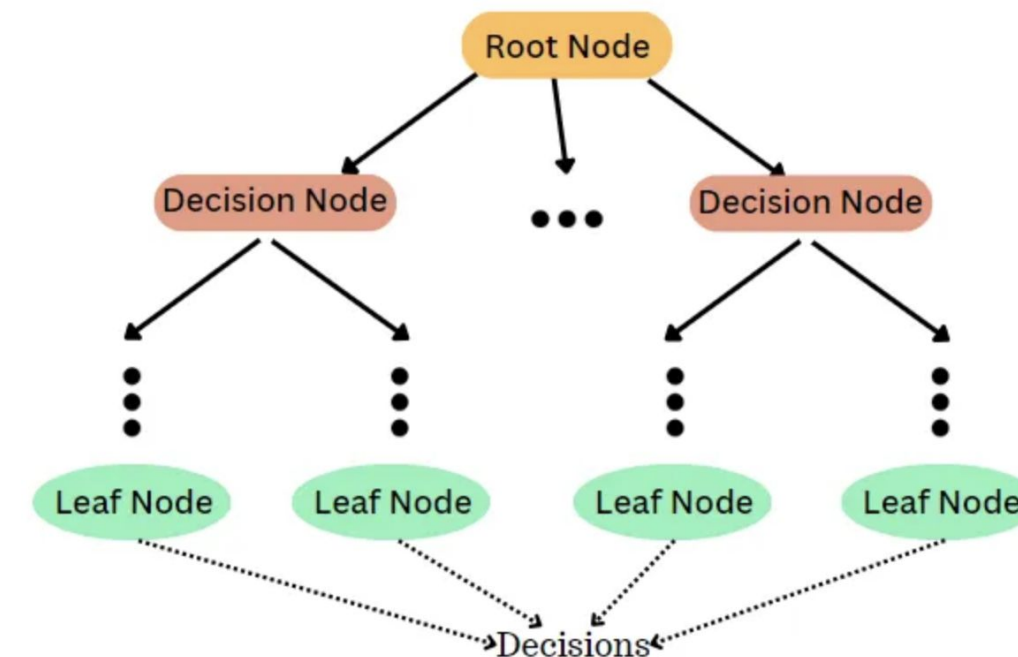
❖ 결정 트리(Decision Tree)란?

- ✓ 학습을 통해 데이터에 존재하는 규칙(Condition)을 발견하고, 이를 트리 형태로 구성

루트 노드(Root Node): 데이터의 첫 번째 분할 기준(조건)을 결정

결정 노드 (Decision Node) : 데이터를 더 작은 그룹으로 나누는 추가적인 조건을 포함

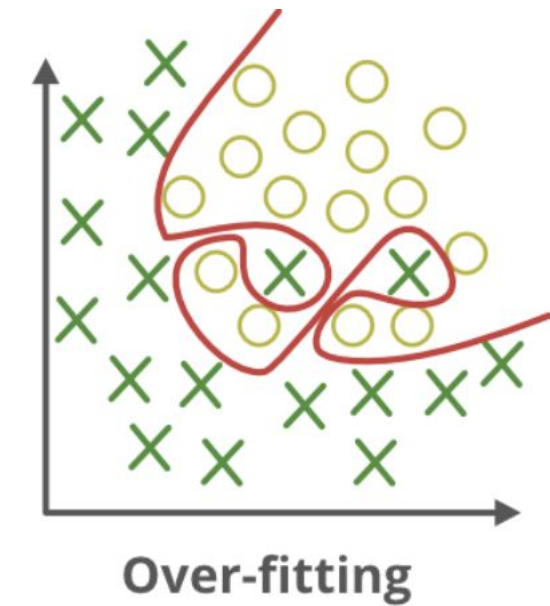
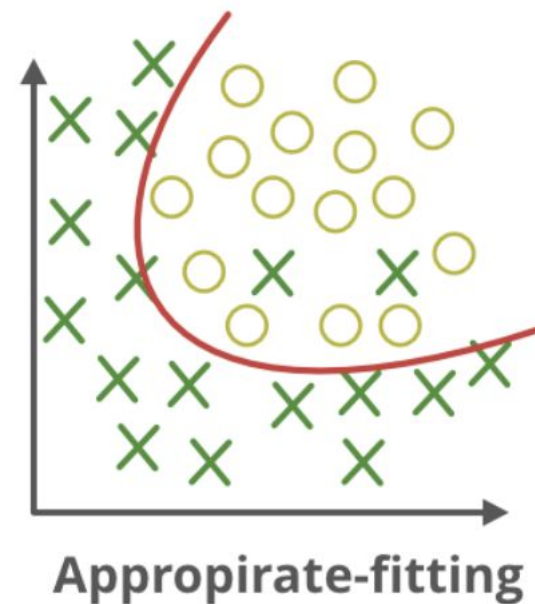
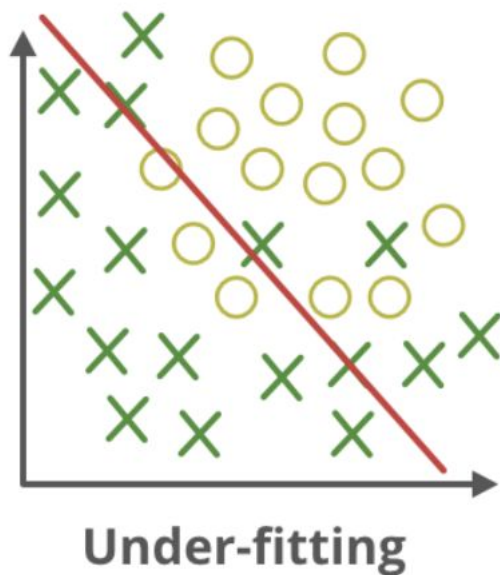
리프 노드(Leaf Node): 최종적으로 분류된 클래스(예측값)를 나타냄



#1.2 결정 트리(Decision Tree)

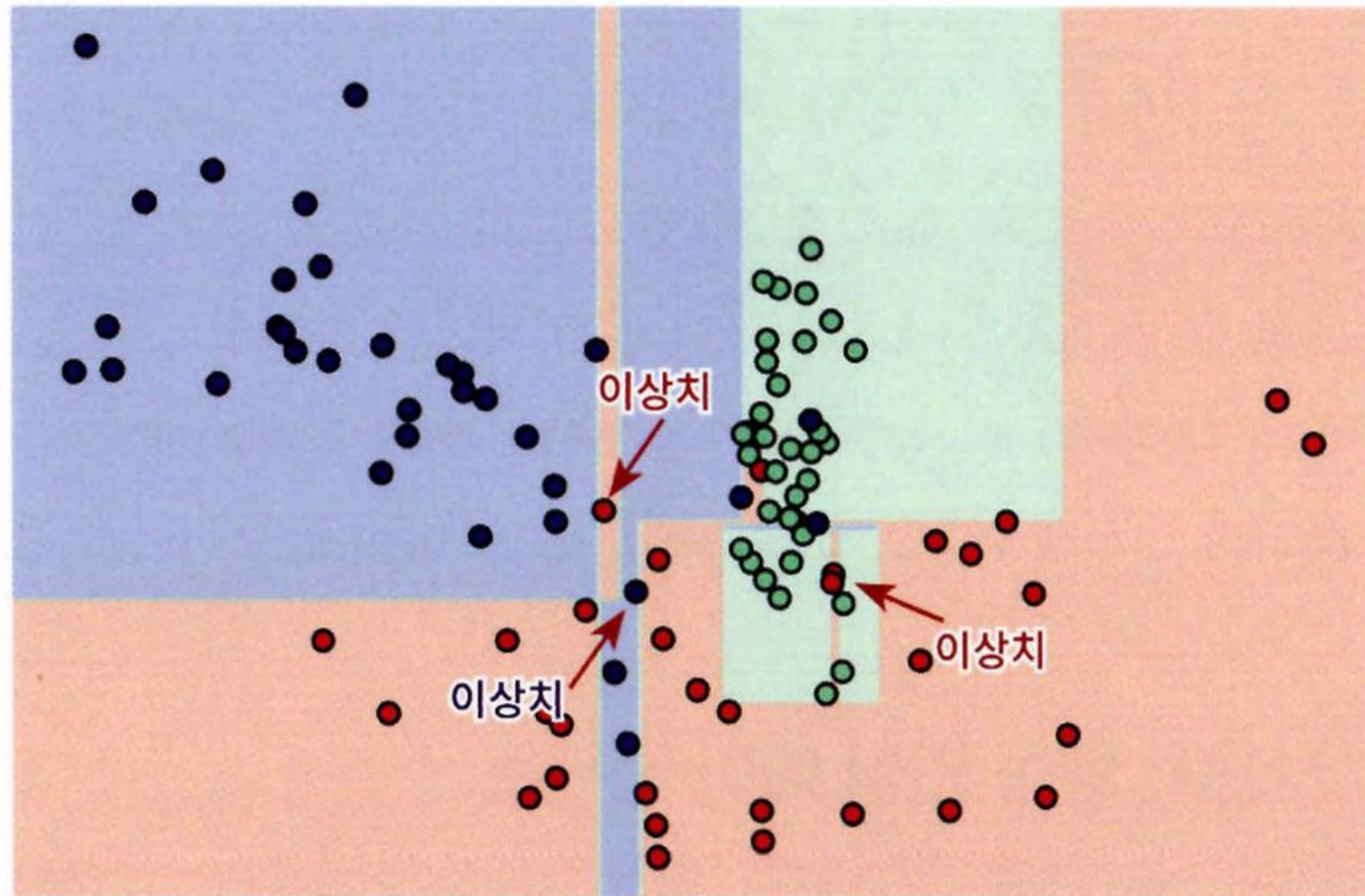
◆ 결정 트리 규칙 선택의 중요성

- ✓ 규칙 선택 기준에 따라 모델의 성능과 일반화 능력에 큰 영향
- ✓ 데이터의 어떤 기준(속성, 특징 등)을 바탕으로 규칙을 설정하는지가 매우 중요
- ✓ 너무 많은 규칙 생성
 - 과적합(Overfitting) 발생 가능
 - 학습 데이터는 잘 맞추나 새로운 데이터 예측력이 떨어짐

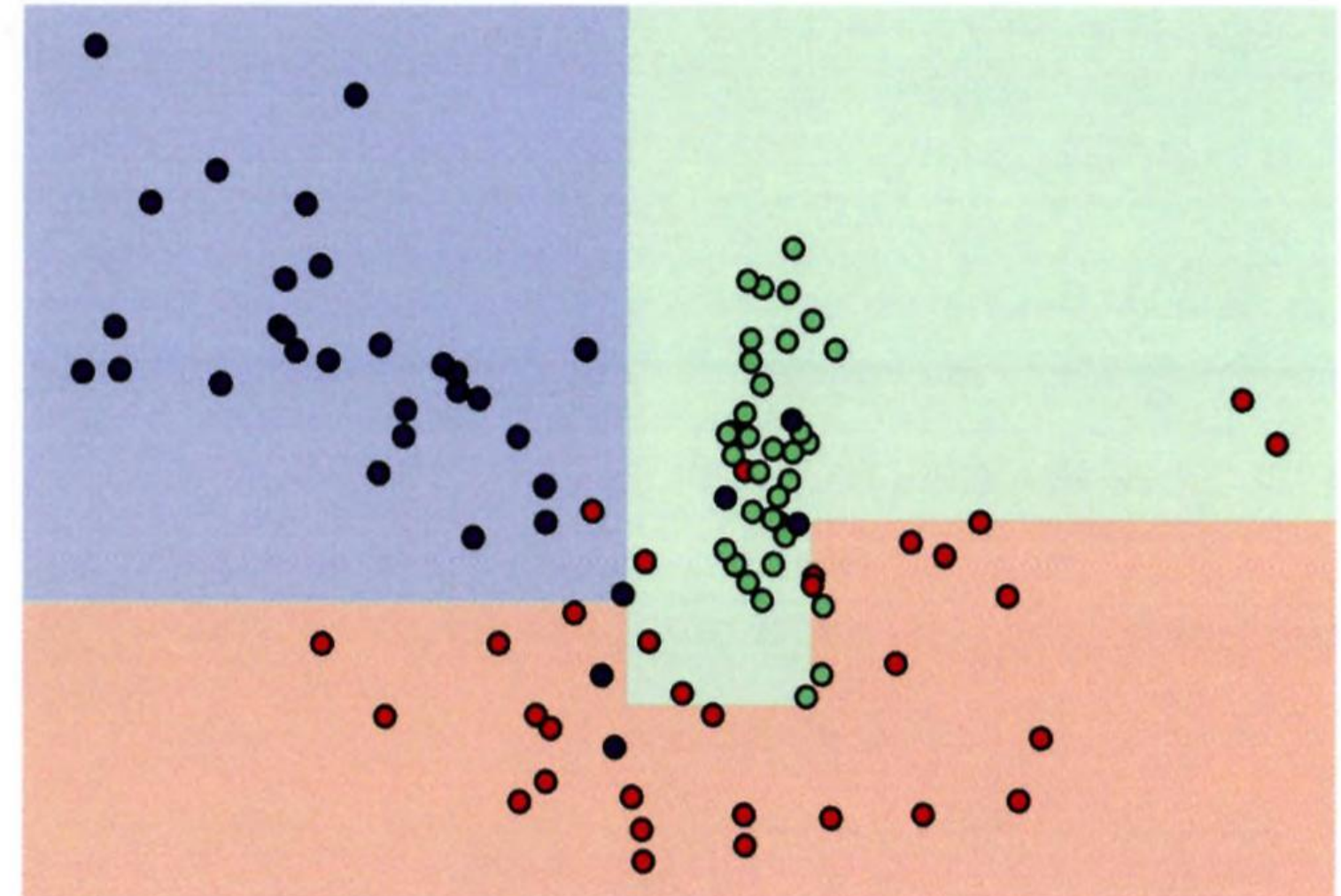


#1.2 결정 트리(Decision Tree)

◆ 과적합 (Overfitting)



과적합 O

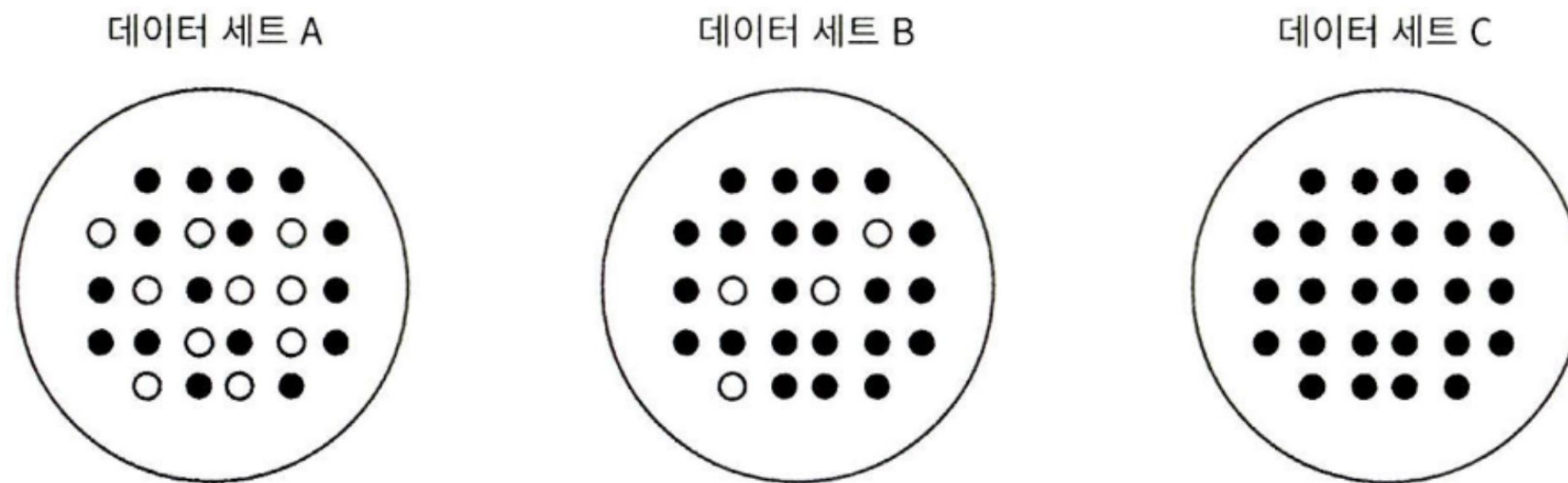


과적합 X

#1.2 결정 트리(Decision Tree)

◆ 정확도를 높이는 결정 트리 구성 전략

- ✓ 많은 데이터가 포함될 수 있는 규칙(Condition)으로 데이터를 분류하는 것이 중요
- ✓ 데이터를 최대한 균일한 그룹으로 나누도록 트리를 분할(Split)
- ✓ 균일한 데이터 세트란?
 - 하나의 규칙으로 분류된 데이터가 대부분 동일한 클래스에 속하는 경우



순서: C, B,
A

#1.2 결정 트리(Decision Tree)

◆ 결정 트리의 분할 기준 계산법

✓ 결정 트리는 어떤 규칙으로 데이터를 나눌지 결정하기 위해 다음 두 가지 지표를 사용:

□ 정보 이득 지수 (Information Gain)

- 엔트로피(Entropy)를 기반으로 한 측정법
- 클수록 좋은 분할 기준

□ 지니 계수 (Gini Index)

- 데이터가 얼마나 혼합되어 있는지를 측정
- 값이 작을수록 균일한 그룹을 나타냄
- 사이킷런의 `DecisionTreeClassifier`는 기본적으로 지니 계수(Gini Index)를 사용해 데이터 분할

◆ 결정 트리의 장점과 단점

★ 장점

- ✓ 균일도 기반
 - 이해하기 쉽고, 직관적
- ✓ 사전 가공(Preprocessing)의 영향이 적음
 - 피처 스케일링(Scaling), 정규화(Normalization) 불필요

! 단점

- ✓ 정확도를 높이기 위해 조건을 추가하면 트리의 깊이가 계속 커짐
 - 과적합(Overfitting) 문제 발생
 - 새로운 데이터에 대한 예측 정확도 하락

#1.2 결정 트리(Decision Tree)

❖ 결정 트리 구조의 시각화

✓ 주로 **Graphviz**를 이용하여 수행

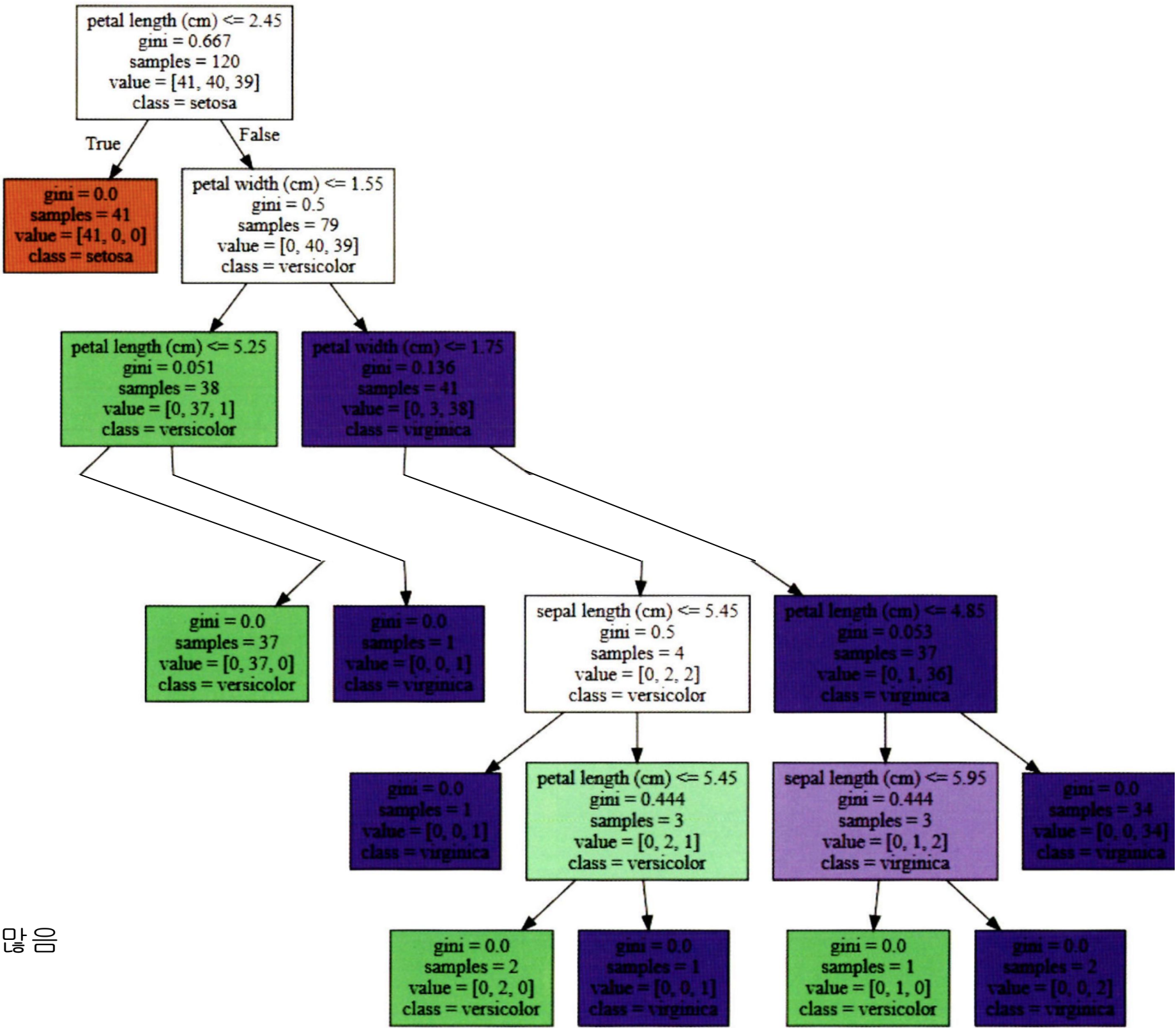
```
# Graphviz로 시각화
from sklearn.tree import export_graphviz
import graphviz

export_graphviz(dt_clf,
                out_file="tree.dot",
                class_names=iris_data.target_names,
                feature_names=iris_data.feature_names,
                impurity=True,
                filled=True)

with open("tree.dot") as f:
    dot_graph = f.read()
graphviz.Source(dot_graph)
```

petal length(cm) <=2.45: 자식 노드 생성 규칙 조건,
이 조건이 없으면 리프 노드

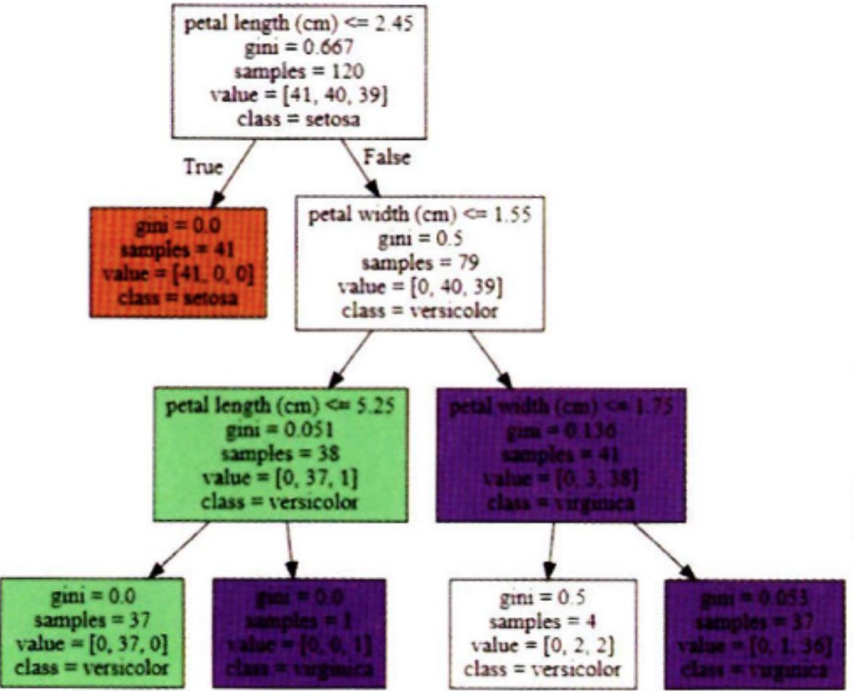
gini = x: 데이터 분포 지니 계수
samples = x: 데이터 건수
value = []: 클래스 기반 데이터 건수
class = x: 하위 노드를 가질 경우 클래스 **x**의 개수가 가장 많음



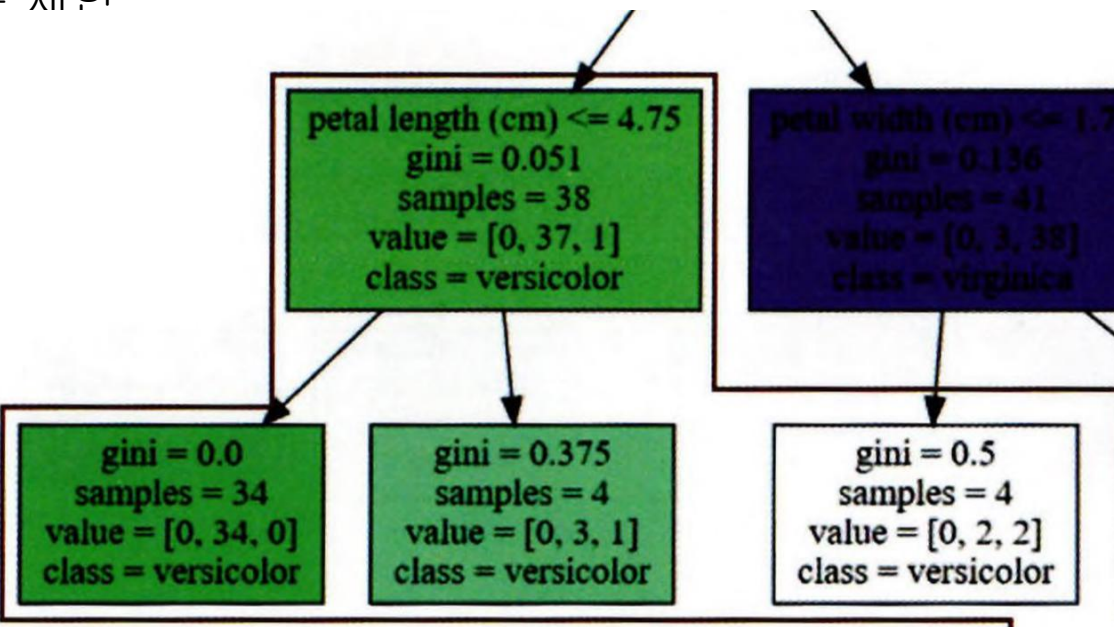
#1.2 결정 트리(Decision Tree)

◆ 결정 트리 주요 파라미터 (Parameters)

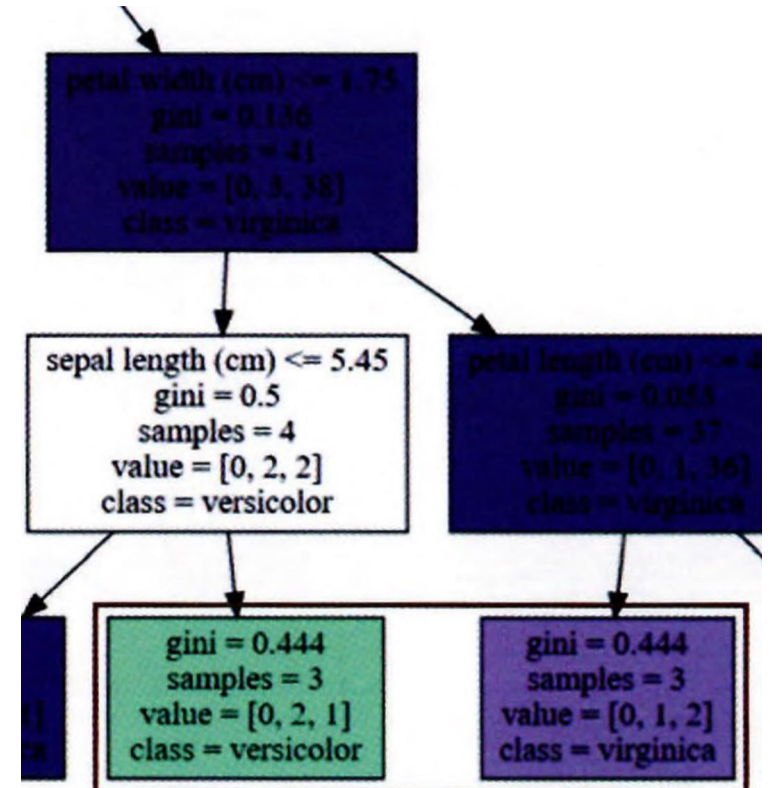
- ① min_samples_split
 - ✓ 노드를 분할하기 위한 최소 샘플 개수
 - ✓ 이 개수보다 적으면 더 이상 분할하지 않음
- ② min_samples_leaf
 - ✓ 리프 노드가 가져야 하는 최소 샘플 개수
 - ✓ 작게 설정하면 과적합 가능성 증가
- ③ max_features
 - ✓ 최적의 분할을 위해 고려할 최대 Feature의 개수
 - ✓ 많이 성능은 좋아지지만 과적합 위험 증가
- ④ max_depth
 - ✓ 트리의 최대 깊이 제한
 - ✓ 너무 깊으면 과적합 발생 가능성 증가
- ⑤ max_leaf_nodes
 - ✓ 리프 노드의 최대 개수 제한



max_depth = 3



min_samples_leaf = 4



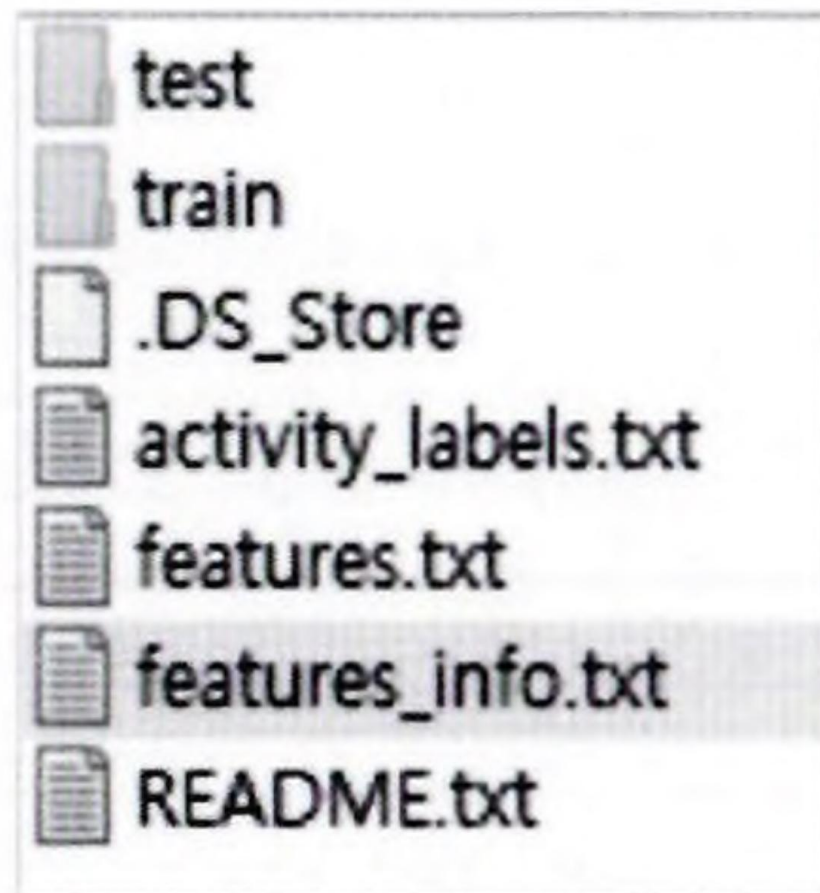
min_samples_split = 4

#1.2 결정 트리(Decision Tree)

❖ 사용자 행동 인식 데이터 세트 (Human Activity Recognition Dataset)

❖ 데이터 세트 개요

- ✓ 출처: UCI Machine Learning Repository
- ✓ 목적: 스마트폰 센서를 활용해 사람의 동작을 인식 및 분류하는 머신러닝 데이터
- ✓ 대상: 총 30명의 실험 참여자



#1.2 결정 트리(Decision Tree)

❖ 모델 학습(디폴트 하이퍼 파라미터)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

dt_clf = DecisionTreeClassifier(random_state=156)
# Train
dt_clf.fit(X_train , y_train)
# Predict
pred = dt_clf.predict(X_test)
```

pred: 0.8548

❖ GridSearchCV로 max_depth 값 변화에 따른 성능 측정

```
from sklearn.model_selection import GridSearchCV

params = {
    'max_depth' : [ 6, 8 ,10, 12, 16 ,20, 24]
}

grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5, verbose=1 )
grid_cv.fit(X_train , y_train)

# DataFrame
cv_results_df = pd.DataFrame(grid_cv.cv_results_)
cv_results_df[['param_max_depth', 'mean_test_score']]
```

	param_max_depth	mean_test_score
0	6	0.850791
1	8	0.851069
2	10	0.851209
3	12	0.844135
4	16	0.851344
5	20	0.850800
6	24	0.849440

param_max_depth : 8 -> highest

#1.2 결정 트리(Decision Tree)

❖ max_depth, min_samples_split 값 변화에 따른 성능 측정

```
params = {  
    'max_depth' : [ 8 , 12, 16 ,20],  
    'min_samples_split' : [16, 24],  
}  
  
grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5, verbose=1 )  
grid_cv.fit(X_train , y_train)  
print('pred: {0:.4f}'.format(grid_cv.best_score_))  
print('최적 하이퍼 파라미터:', grid_cv.best_params_)
```

pred: 0.8549

최적 하이퍼 파라미터: {'max_depth': 8, 'min_samples_split': 16}

❖ 위에 최적 하이퍼 파라미터 값을 가지고 예측 수행

```
best_df_clf = grid_cv.best_estimator_  
pred1 = best_df_clf.predict(X_test)  
accuracy = accuracy_score(y_test , pred1)  
print('pred:{0:.4f}'.format(accuracy))
```

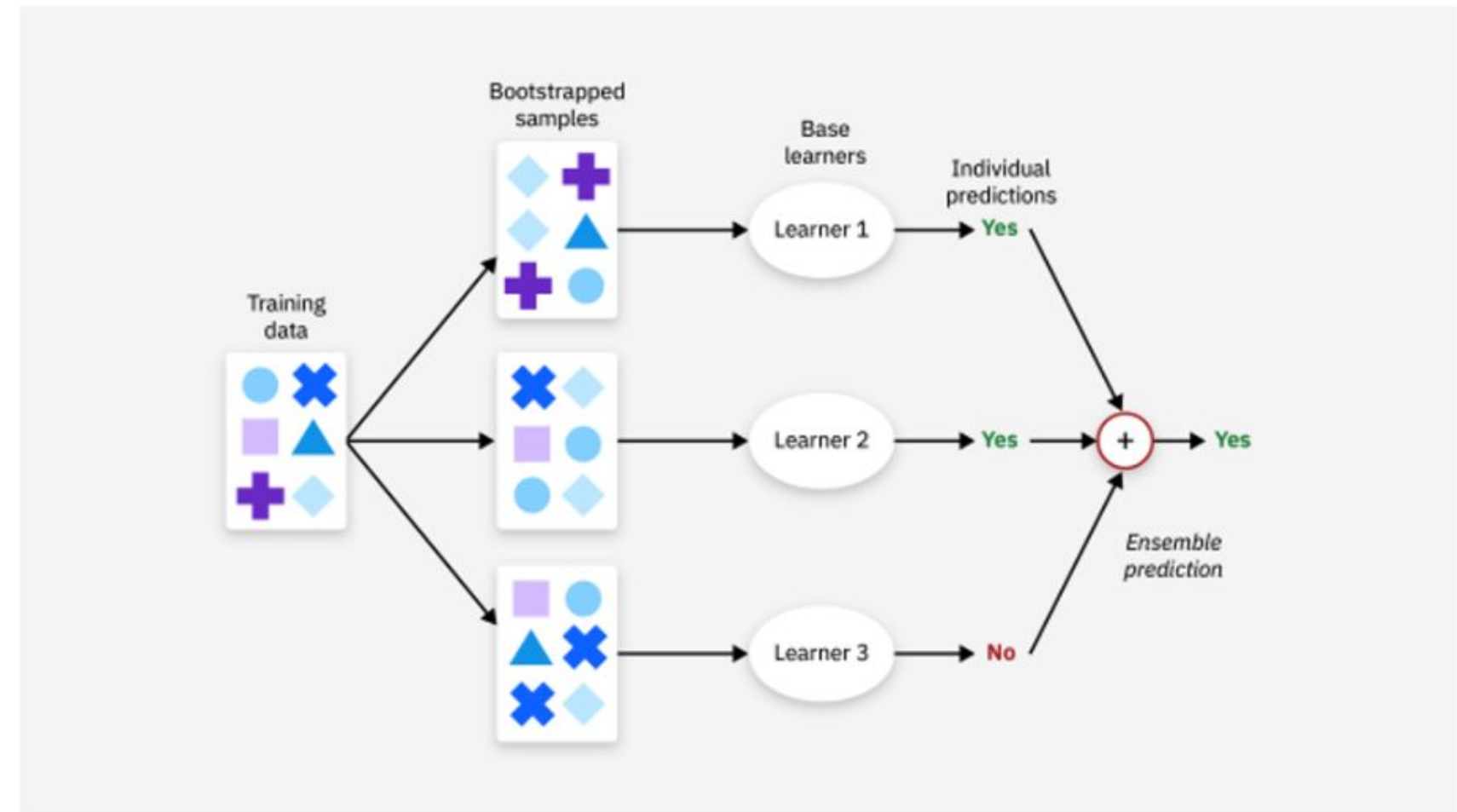
pred: 0.8717

4.3 앙상블 학습



#4.3.1 앙상블 학습 개요

- **앙상블 학습:** 여러 개의 분류기를 생성하고 그 예측을 결합함으로써 보다 정확한 최종 예측을 도출하는 기법
- 목표: 단일 분류기보다 신뢰성이 높은 예측값을 얻는 것
- ex) 랜덤 포레스트, 그래디언트 부스팅 알고리즘, XGBoost, LightGBM, 스태킹
- 유형: 보팅(Voting), 배깅(Bagging), 부스팅(Boosting), 스태킹



#4.3.1 앙상블 학습 개요

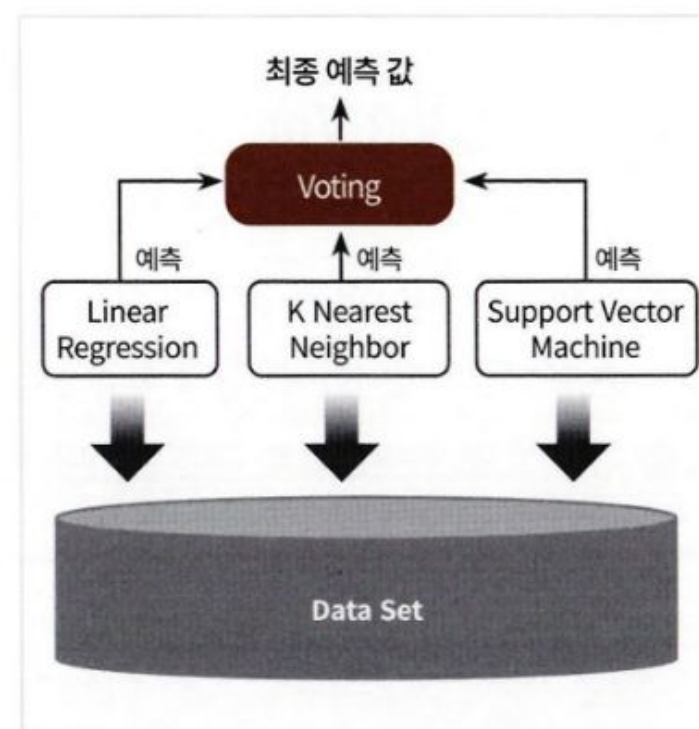
✓ 보팅(Voting) vs 배깅(Bagging)

- 공통점

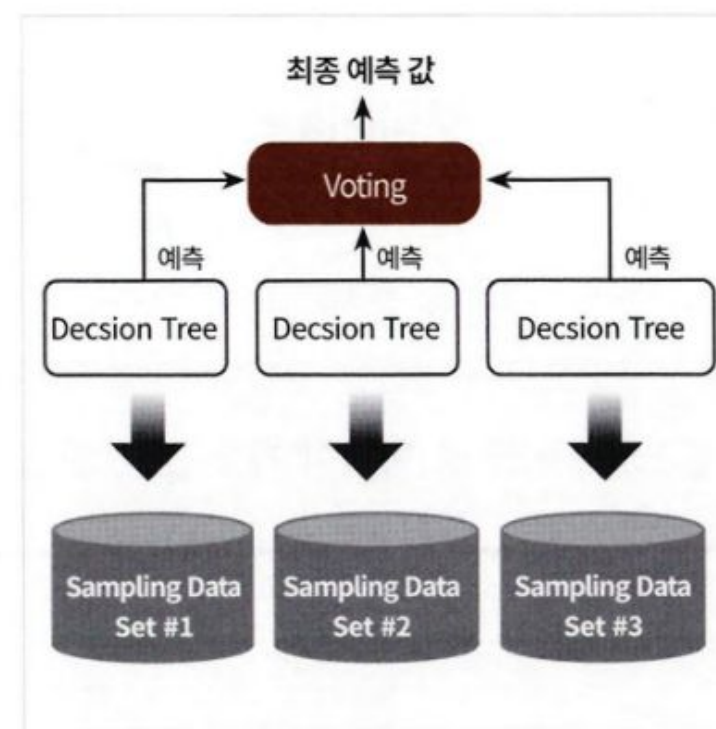
여러 개의 분류기가 투표를 통해 최종 예측 결과를 결정

- 차이점

보팅은 서로 다른 알고리즘을 가진 분류기 결합
배깅은 데이터 샘플링을 다르게 학습 수행



Voting 방식



Bagging 방식

부트스트래핑!! (Bootstrapping):
중복을 허용하여 원본 데이터에서 랜덤하게 샘플을 뽑는 방식

#4.3.1 앙상블 학습 개요

✓ 부스팅(Boosting) & 스택킹

• 부스팅

여러개의 분류기가 순차적으로 학습
앞에서 학습한 분류기가 예측이 틀린 데이터에
대해서 다음 분류기에 가중치(weight) 부여
ex) 그래디언트 부스트, XGBoost, LightGBM

• 스택킹

여러가지 다른 모델의 예측 결과값을 다시 학습
데이터로 만들어서 다른 모델(메타 모델)로
재학습 시켜 결과를 예측하는 방법

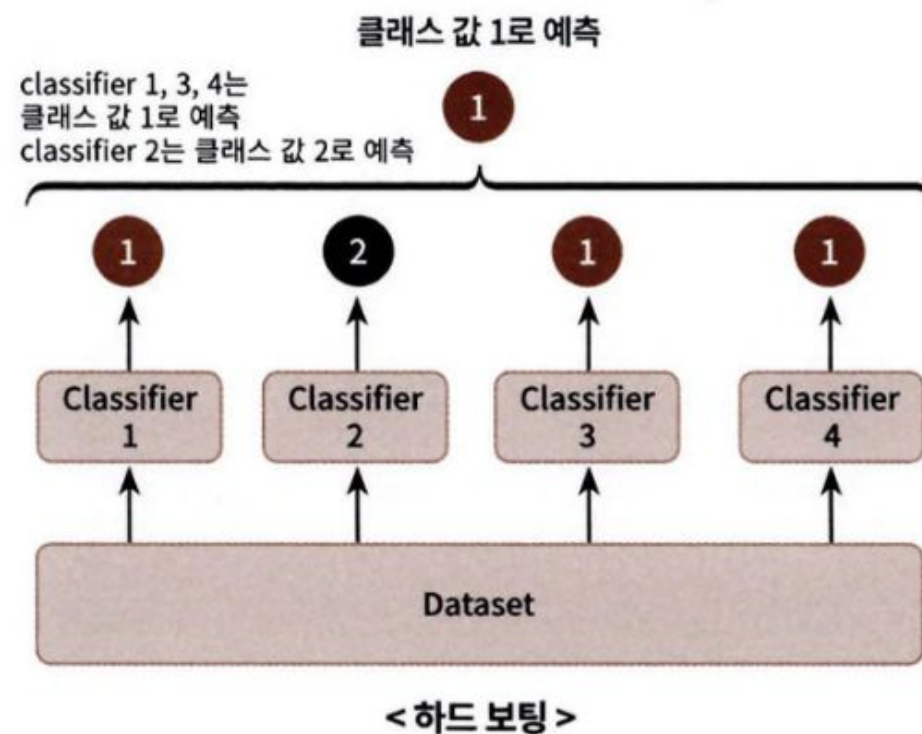
#4.3.2 보팅 유형_하드 보팅과 소프트 보팅

✓ 하드 보팅 vs 소프트 보팅

• 하드 보팅(Hard Voting)

예측한 결과값들 중 다수의 분류기가 결정한
예측값을 최종 보팅 결과값으로 선정하는 방식

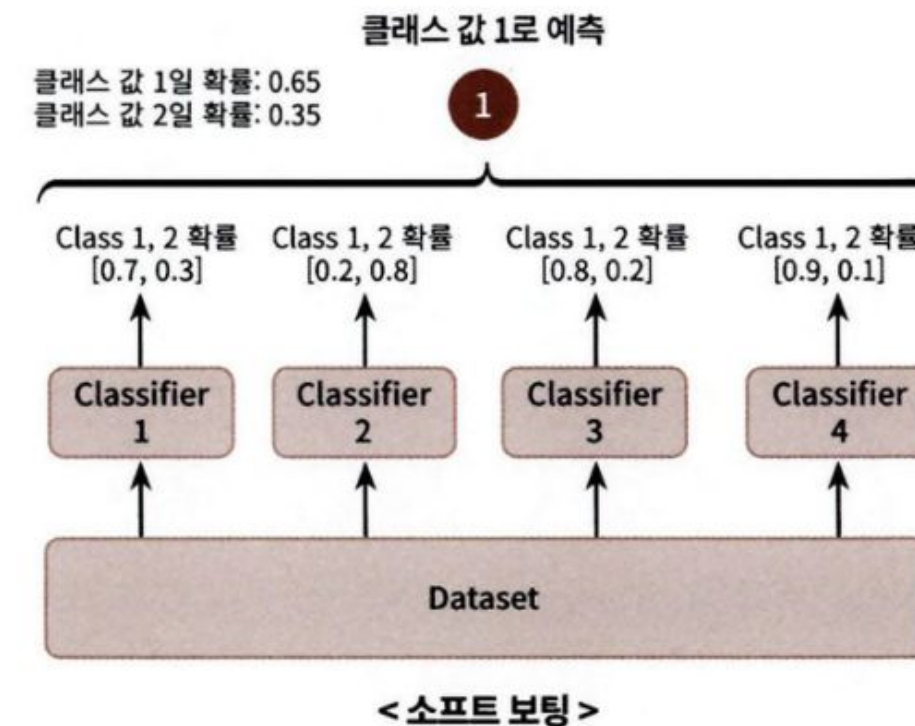
Hard Voting은 다수의 classifier 간 다수결로 최종 class 결정



• 소프트 보팅(Soft Voting)

분류기들의 레이블 값 결정 확률을 모두 더하고
이를 평균해서 이들 중 확률이 가장 높은 레이블
값을 최종 보팅 결과값으로 선정하는 방식

Soft Voting은 다수의 classifier들의 class 확률을 평균하여 결정



#4.3.3 보팅 분류기

✓ 사이킷런 VotingClassifier

- 주요 인자: estimators, voting
- estimators: 리스트 값, 보팅에 사용될 여러 개의 Classifier 객체들을 튜플 형식으로 입력 받음
- voting = 'hard', 'soft'

▼ 필요한 모듈과 데이터 로딩

```
import pandas as pd

from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer = load_breast_cancer()

data_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
data_df.head(3)
```


#4.3.3 보팅 분류기

✓ 사이킷런 VotingClassifier

- 로지스틱 회귀와 KNN을 기반으로 소프트 보팅 방식의 새로운 보팅 분류기 생성
- 보팅으로 여러 개의 기반 분류기를 결합한다고 해서 무조건 기반 분류기보다 예측 성능이 ↑ X
- But !! 단일 ML 알고리즘보다 뛰어난 예측 성능 갖는 경우가 많음

```
# 개별 모델은 로지스틱 회귀와 KNN임.
lr_clf = LogisticRegression(solver='liblinear')
knn_clf = KNeighborsClassifier(n_neighbors=8)

# 개별 모델을 소프트 보팅 기반의 앙상블 모델로 구현한 분류기
vo_clf = VotingClassifier(estimators=[('LR', lr_clf), ('KNN', knn_clf)], voting='soft')

X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target, test_size=0.2, random_state=156)

# VotingClassifier 학습/예측/평가
vo_clf.fit(X_train, y_train)
pred = vo_clf.predict(X_test)
print('Voting 분류기 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))

# 개별 모델의 학습/예측/평가
classifiers = [lr_clf, knn_clf]
for classifier in classifiers:
    classifier.fit(X_train, y_train)
    pred = classifier.predict(X_test)
    class_name = classifier.__class__.__name__
    print('{0} 정확도: {1:.4f}'.format(class_name, accuracy_score(y_test, pred)))

Voting 분류기 정확도: 0.9561
LogisticRegression 정확도: 0.9474
KNeighborsClassifier 정확도: 0.9386
```

!! 결정 트리 알고리즘의 장점은 그대로 취하고 단점은 보완하며 편향-분산 트레이드 오프 효과를 극대화

배깅과 페이스팅



배깅과 페이스팅

✓ 배깅 & 페이스팅

- 배깅(bagging, bootstrap aggregating)
훈련 세트에서 중복을 허용하여 샘플링
- 페이스팅(pasting)
중복을 허용하지 않고 샘플링

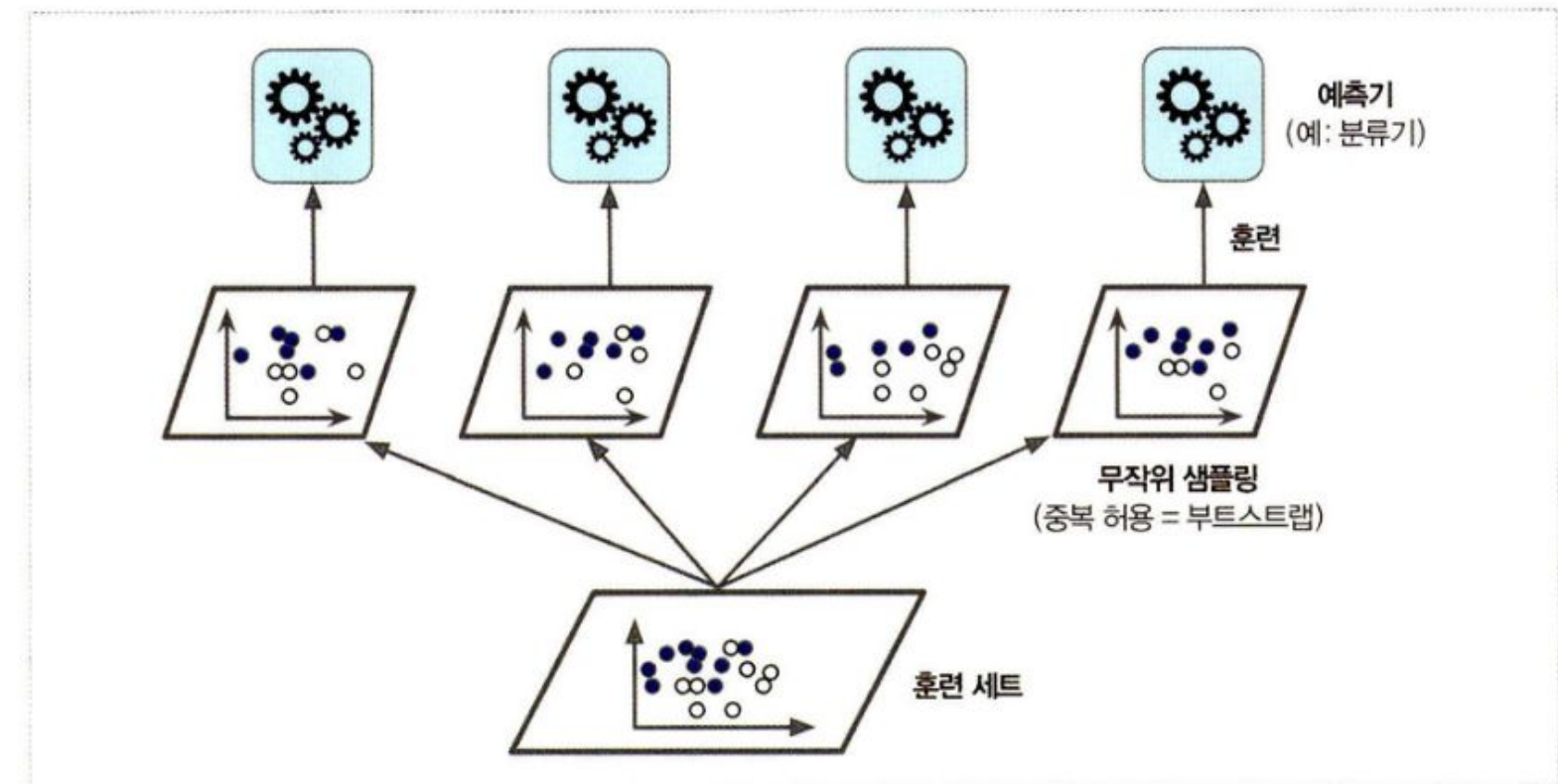


그림 7-4 배깅과 페이스팅은 훈련 세트에서 무작위로 샘플링하여 여러 개의 예측기를 훈련합니다.

배깅과 페이스팅

✓ 배깅

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500, max_samples=100, bootstrap=True, n_jobs=-1)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

- Bootstrap = True → 중복 허용
- n_jobs : 사이킷런이 훈련과 예측에 사용할 CPU 코어 수를 지정
- -1 → 가용한 모든 코어 사용

배깅과 페이스팅

✓ 결정 경계 비교

- 앙상블의 예측이 일반화가 더 잘됨
오차 수는 비슷하지만 결정 경계 덜 불규칙
→ 앙상블은 비슷한 편향에서 더 작은 분산을 만들

- 편향: 배깅 > 페이스팅
!! but 분산 ↓ → 배깅 선호

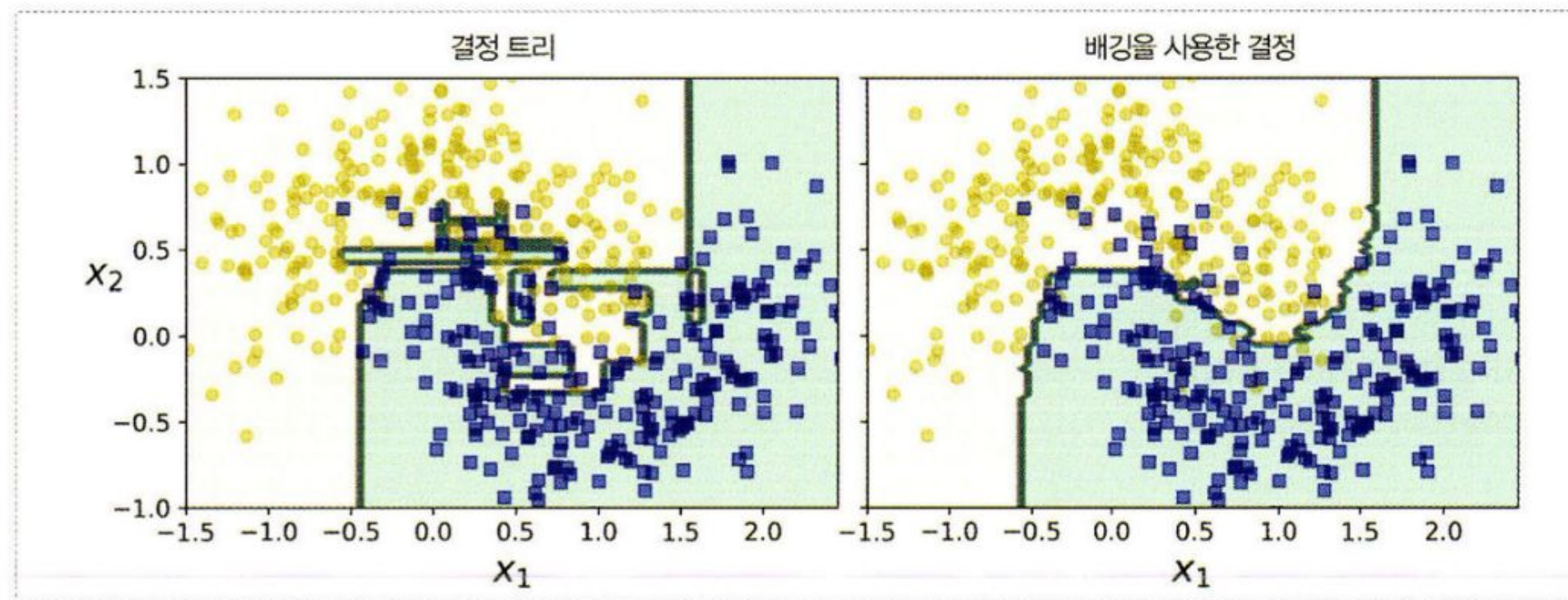


그림 7-5 단일 결정 트리(왼쪽)와 500개 트리로 만든 배깅 앙상블(오른쪽) 비교

oob 평가

✓ oob(out-of-bag)

- 배깅 사용시 어떤 샘플은 중복 샘플링되고 어떤 샘플은 선택되지 않을 수 있음
- 선택되지 않은 훈련 샘플의 나머지: oob 샘플
- 별도의 검증 세트 없이 oob 샘플로 예측기 평가 가능

oob 평가

✓ oob(out-of-bag)

- oob_score=True 지정
- 훈련이 끝난 후 자동으로 oob 평가 수행
- 평가 점수 결과 oob_score_ 변수에 저장

```
bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500, bootstrap=True, n_jobs=-1, oob_score=True)
```

```
bag_clf.fit(X_train, y_train)  
bag_clf.oob_score_
```

```
0.9582417582417583
```

```
from sklearn.metrics import accuracy_score  
y_pred = bag_clf.predict(X_test)  
accuracy_score(y_test, y_pred)
```

```
0.956140350877193
```

▶ oob 평가 결과

▶ 테스트 세트 평가 결과
→ 비슷함!!

랜덤 패치와 랜덤 서브스페이스

✓ 랜덤 패치 & 랜덤 서브스페이스: 특성 샘플링

- 랜덤 패치
- 데이터 샘플, 특성 둘 다 샘플링
- 랜덤 서브스페이스
- 특성만 샘플링, 샘플 전체 사용

→ 다양한 조합으로 학습하게 하여 편향은 늘어날 수 있지만 분산을 유의미하게 낮춤

고차원 데이터(ex. 이미지)에 유리

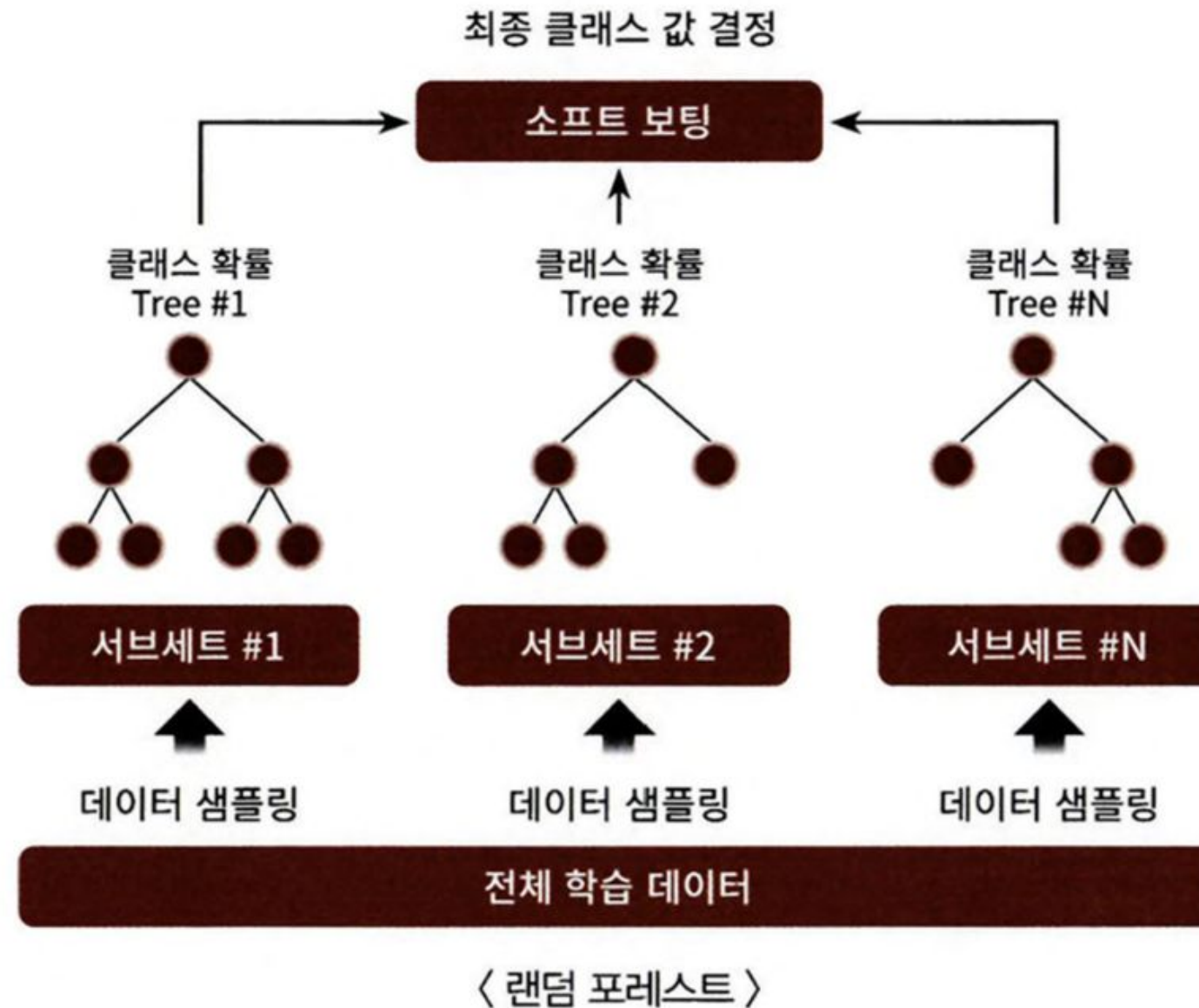
#03 랜덤 포레스트(Random Forest)



#4.1 랜덤 포레스트 개요 및 실습

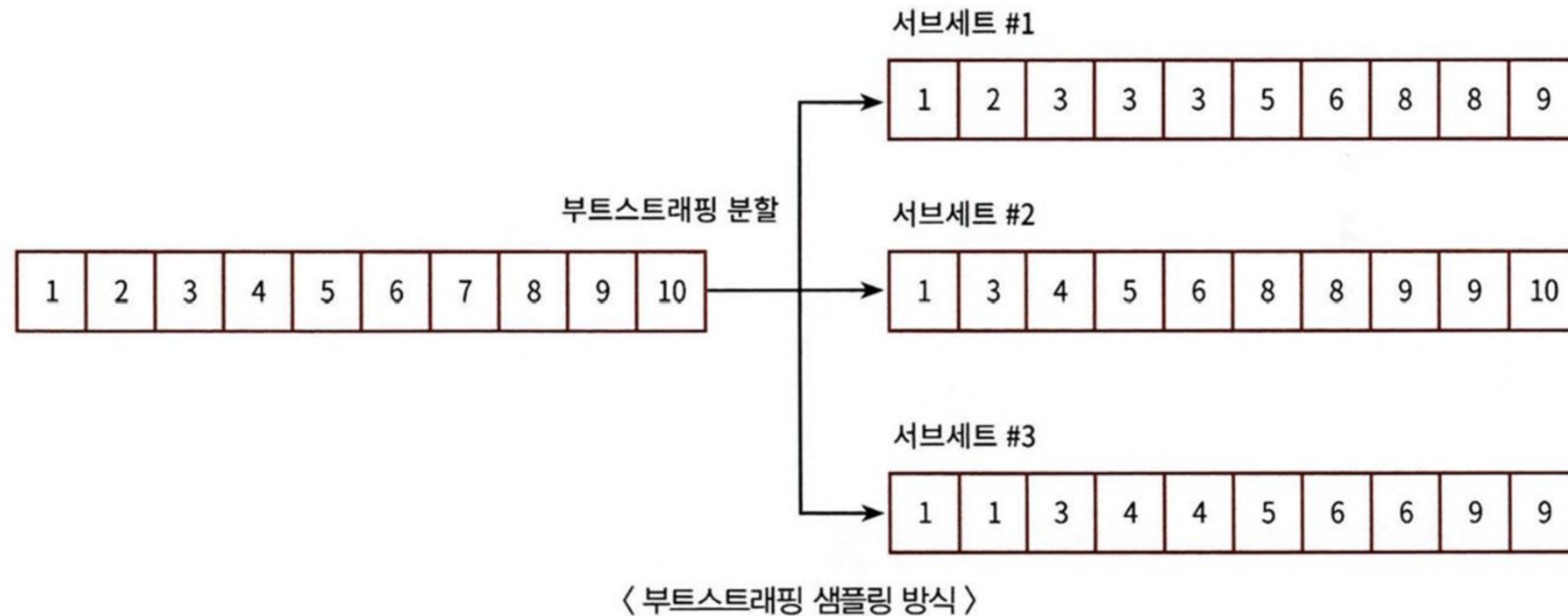
랜덤 포레스트:

배깅(bagging) 방식으로 각자의 데이터를 개별적으로 학습한 뒤, 보팅을 통해 예측 결정



#4.1 랜덤 포레스트 개요 및 실습

부트스트래핑: 여러 개의 데이터 세트를 중첩되게 분리



#4.2 랜덤 포레스트 하이퍼 파라미터 및 튜닝

랜덤 포레스트의 파라미터

n_estimators	결정 트리의 개수 지정, 디폴트 10개
max_features	결정 트리에서 사용된 파라미터와 동일 (단, 기본 'sqrt')
max_depth min_samples_leaf min_samples_split	결정 트리에서 과적합을 개선하기 위해 사용되는 파라미터와 동일

#4.2 랜덤 포레스트 하이퍼 파라미터 및 튜닝

```
from sklearn.model_selection import GridSearchCV

params = {
    'max_depth': [8, 16, 24],
    'min_samples_leaf': [1, 6, 12],
    'min_samples_split': [2, 8, 16]
}

# RandomForestClassifier 객체 생성 후 GridSearchCV 수행
rf_clf = RandomForestClassifier(n_estimators=100, random_state=0, n_jobs=-1)
grid_cv = GridSearchCV(rf_clf, param_grid=params, cv=2, n_jobs=-1)
grid_cv.fit(X_train, y_train)

print('최적 하이퍼 파라미터:\n', grid_cv.best_params_)
print('최고 예측 정확도: {0: .4f}'.format(grid_cv.best_score_))
```

GridSearchCV를 이용한
하이퍼 파라미터 튜닝

```
rf_clf1 = RandomForestClassifier(n_estimators=100, min_samples_leaf=6,
                                max_depth=16, min_samples_split=2, random_state=0)
rf_clf1.fit(X_train, y_train)
pred = rf_clf1.predict(X_test)
print('예측 정확도: {0: .4f}'.format(accuracy_score(y_test, pred)))
```

RandomForestClassifier을
학습시킨 뒤 테스트 데이터 세트
에서 예측 성능 측정

#4.3 서포트 벡터 머신(SVM)

주요 분류 알고리즘 중 하나

분류 이외에도 선형, 회귀, 이상치 탐색 등에 사용 가능

특히 복잡한 분류에 잘 들어맞으며, 작거나 중간 크기의 데이터셋에 적합

#4.3.1 선형 SVM 분류

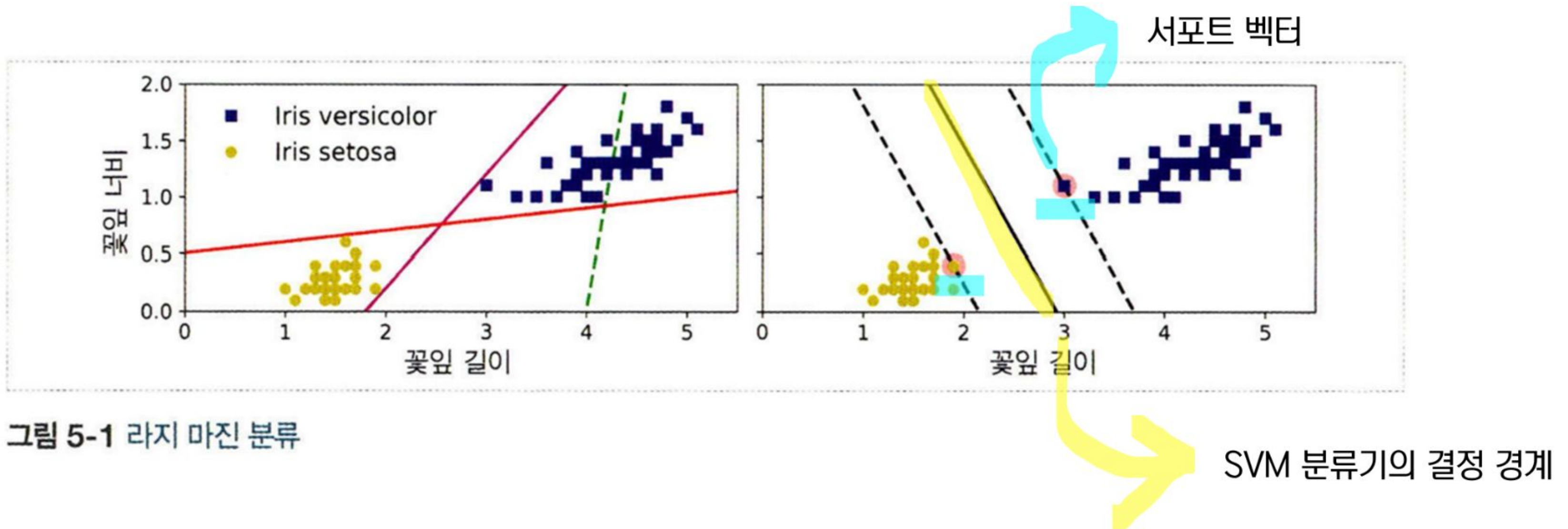


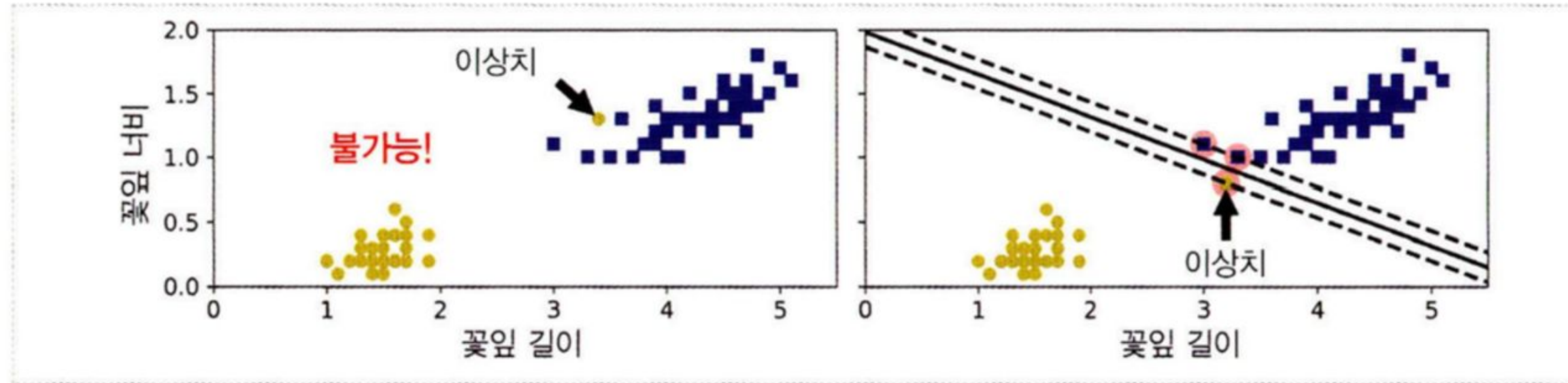
그림 5-1 라지 마진 분류

라지 마진 분류

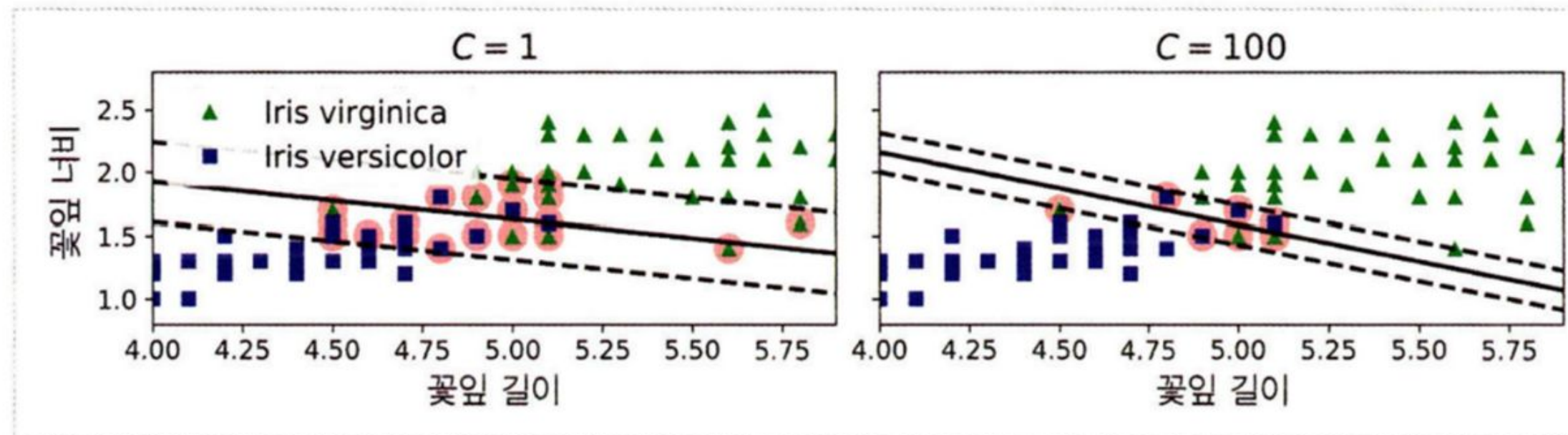
- : SVM 분류기를 클래스 사이 가장 폭이 넓은 지점에 찾는 것
- : 하드 마진과 소프트 마진으로 나누어짐

#4.3.1 선형 SVM 분류

하드 마진 분류 : 데이터가 선형적으로 구분되어야만 제대로 작동, 이상치에 민감하다는 단점

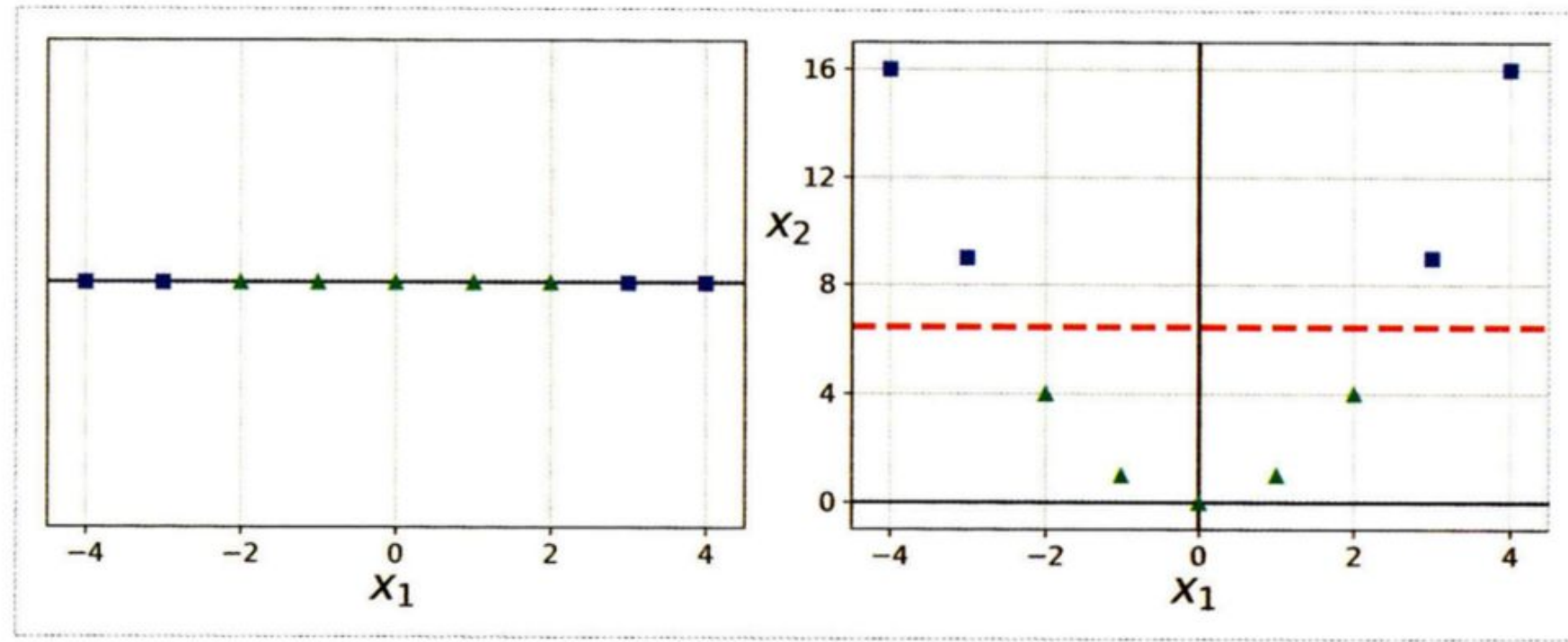


소프트 마진 분류 : 도로 폭 유지와 마진 오류 사이의 적절한 균형



#4.3.2 비선형 SVM 분류

비선형 데이터셋을 다루기 위해서는 특성을 추가해야 함



⇒ 문제점:

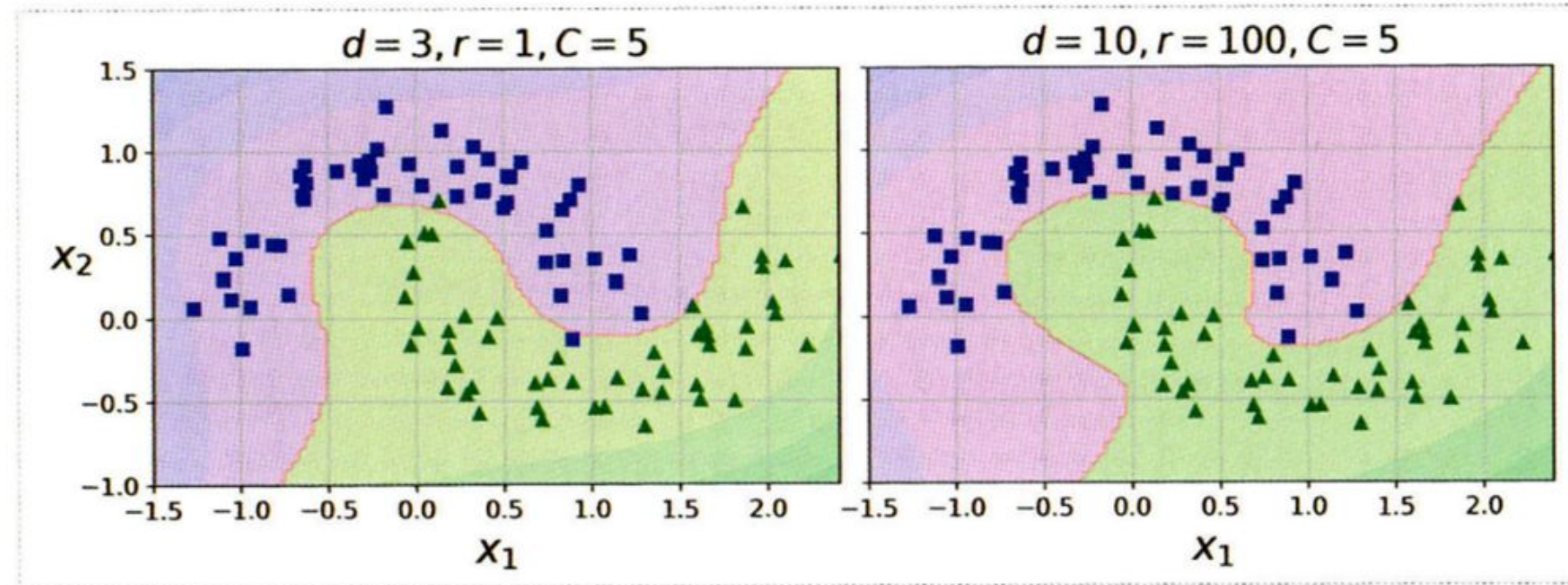
낮은 차수의 다항식으로 복잡한 데이터셋을 표현하기 어려움
높은 차수의 다항식은 많은 특성을 추가해 모델을 느리게 만들

#4.3.2 비선형 SVM 분류

해결1 - 커널 트릭

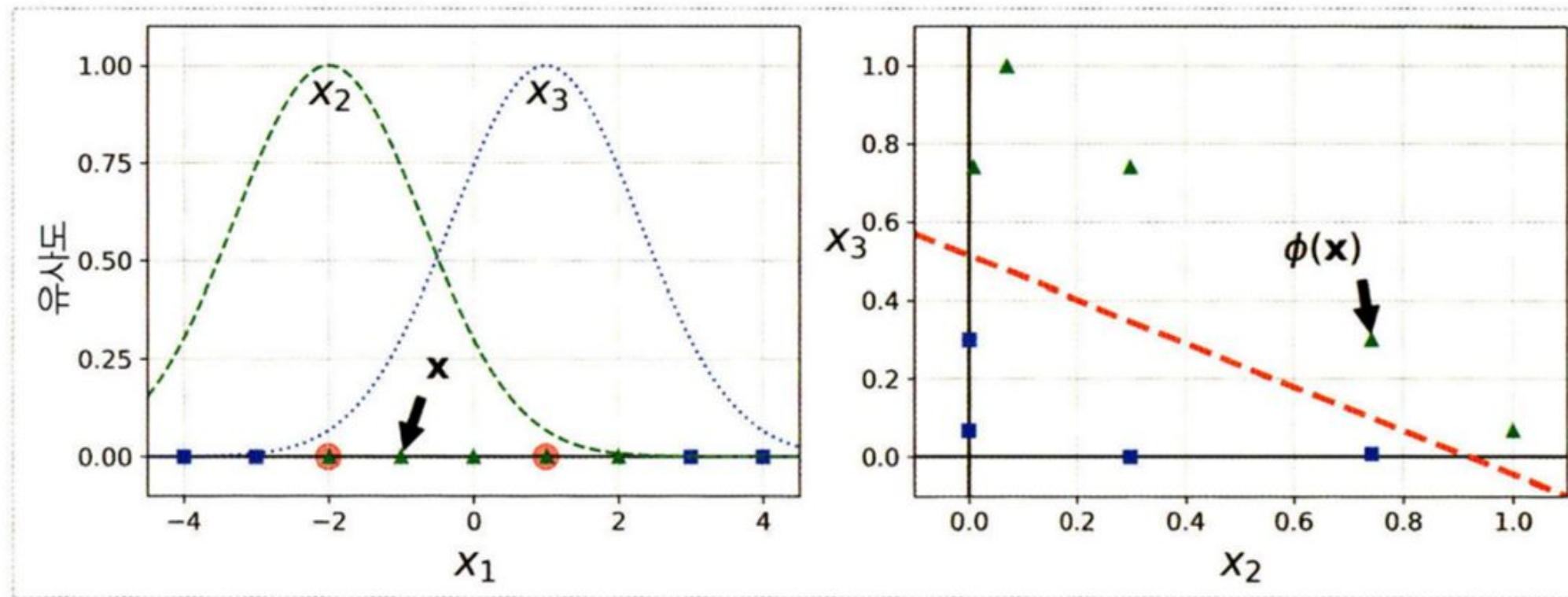
: 실제로는 특성을 추가하지 않으면서, 다항식 특성을 많이 추가한 것과 같은 결과를 만듦

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
])
poly_kernel_svm_clf.fit(X, y)
```



#4.3.2 비선형 SVM 분류

해결2 - 유사도 특성 추가



$$\phi_{\gamma}(\mathbf{x}, \ell) = \exp\left(-\gamma \|\mathbf{x} - \ell\|^2\right)$$

```
rbf_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))
])
rbf_kernel_svm_clf.fit(X, y)
```


THANK YOU

